Embedded Operating System HW02 <A111193>

Part 1: Extend kernel module into a character driver

1. Explain the detailed implementation of your approach.

```
1 #include <linux/module.h>
2 #include <linux/kernel.h>
3 #include <linux/fs.h>
4 #include <linux/cdev.h>
5 #include <linux/uaccess.h>
6 #include <linux/init.h>
7 #include <linux/io.h>
8
9 MODULE_LICENSE("GPL");
10 MODULE_DESCRIPTION("Driver Demo");
11 #define DRIVER_NAME "SenTsai"
12
13 static char buffer[256];
14 static dev_t demo_dev;
15 static struct cdev demo_cdev;
16 static dev_t demo_dev = MKDEV(90, 40);
17
18 static ssize_t demo_read(struct file *f, char __user *buf, size_t len, loff_t *off)
19 {
20         int ret = 0;
21         ret = copy_to_user(buf, buffer, len);
22
23         if(ret == 0)
24         {
25                 printk(KERN_INFO"Read success!\n");
26                 return 0;
27         }else
28         {
29                 printk(KERN_INFO"Read fail!\n");
30                 return -EFAULT;
31         }
32 }
33
34 static ssize_t demo_write(struct file *f, const char __user *buf, size_t len, loff_t *off)
35 {
36         int ret = 0;
37         ret = copy_from_user(buffer, buf, len);
38         if(ret == 0)
39         {
40                 printk(KERN_INFO"Write success!\n");
41                 return 0;
42         }else
43         {
44                 printk(KERN_INFO"Write fail!\n");
45                 return -EFAULT;
46         }
47 }
```

11 行定義 Driver 的名稱，名為 SenTsai

13-16 行為定義 driver 的一些重要變數，buffer 是 kernel space 的緩存，且定義設備號 demo_dev，定義字符設備 demo_cdev，且將此結構的主、次設備號分別設為 90、40。

18-32 行為定義 driver 的一個函式 demo_read，功能是讀取寫入 kernel space 的字串，首先會先宣告一個變數 ret 且初始化為 0，主要是紀錄 copy_to_user()複製時未複製成功的字符數，copy_from_user()會將 buffer 的字串複製到 buf 裡，複製數量是 len，接下來用 if 述句判斷 copy_to_user()未複製成功的字符數，若 ret = 0 代表複製成功，則會印出 Read success!並且回傳 0，若 ret != 0 代表複製失敗，會印出 Read fail!並且回傳-EFAULT 表示失敗。

34-47 行為定義 driver 的另一個函式 demo_write，功能是將字串寫入 kernel space，首先會先宣告一個變數 ret 且初始化為 0，主要是紀錄 copy_from_user()複製時未複製成功的字符數，copy_from_user()會將 buf 的字串複製到 buffer 裡，複製數量是 len，接下來會用 if 述句判斷 copy_from_user()未複製成功的字符數，若 ret = 0 代表複製成功，則會印出 Write success!並且回傳 0，若 ret != 0 代表複製失敗，會印出 Write fail!並且回傳-EFAULT 表示失敗。

```
49 static int demo_open(struct inode *nd, struct file *instance)
50 {
51         printk(KERN_INFO"demo open!!\n");
52         return 0;
53 }
54
55 static int demo_close(struct inode *df, struct file *instance)
56 {
57         printk(KERN_INFO"demo closed!!\n");
58         return 0;
59 }
60
61 static struct file_operations demo_fops =
62 {
63         .owner = THIS_MODULE,
64         .read = demo_read,
65         .write = demo_write,
66         .open = demo_open,
67         .release = demo_close,
68 };
69
70 static int __init demo_init(void)
71 {
72     int ret = 0, error = 0;
73
74     error = register_chrdev_region(demo_dev, 1, DRIVER_NAME);
75     printk("MAJOR = %d MINOR = %d\r\n", MAJOR(demo_dev), MINOR(demo_dev));
76     if (error < 0)
77     {
78         ret = -EBUSY;
79         goto fail;
80     }
81     cdev_init(&demo_cdev, &demo_fops);
82     error = cdev_add(&demo_cdev, demo_dev, 1);
83     if (error < 0)
84     {
85         ret = -EBUSY;
86         goto fail;
87     }
88     printk("Hello demo!!\n");
89     return 0;
90
91
92 fail:
93     return ret;
94 }
95 static void __exit demo_exit(void)
96 {
97         cdev_del(&demo_cdev);
98         unregister_chrdev_region(demo_dev, 1);
99         printk("Goodbye demo!!\n");
100 }
101 module_init(demo_init);
102 module_exit(demo_exit);
```

49-53 行為定義 driver 的一個函式 demo_open，若當檔案成功開啟，印出 demo open!!。

55-59 行為定義 driver 的一個函式 demo_close，若當檔案關閉後，印出 demo close!!。

61-68 行為定義 driver 的公能集 demo_fops，有剛剛上述的功能 read、write、open、close。

70-94 行為當這個 module 被加載進 kernel 之後會呼叫這個函式 demo_init，首先會先宣告兩個變數 ret 跟 error，ret 是函式 demo_init 的回傳值，error 會回傳一個代碼，此代碼為註冊字符設備結果的代號，當註冊成功時會大於等於 0，若失敗則會小於 0，74 行使用 register_chrdev_region()將 demo_dev 的主、次設備號註冊，註冊數量為 1，且名稱是 SenTsai，最後將註冊結果代碼存進 error 中，75 行印出 demo_dev 的主、次設備號，76-80 行處理註冊失敗的情況，會將 ret 設為-EBUSY，且會直接跳到 flag 為 fail 地方執行，也就是 92 行以下的內容，並且直接回傳 ret 這邊值得注意的是，程式碼會遵照上下的排列順序執行，所以 flag 設立的位置要小心，不然會進入無窮迴圈，81 行為初始化字符設備，且添加 demo_fops 的功能，82 行會將剛剛初始化的字符設備加入設備號 demo_dev，數量為 1，同時也會將註冊結果代碼存進 error 中，83-87 行處理處理註冊失敗的情況，會將 ret 設為-EBUSY，且會直接跳到 flag 為 fail 地方執行，最後會印出 Hello demo 並且成功執行這個函式最後回傳 0。

95-100 行為當這個 module 被 kernel 移除之後會呼叫這個函式 demo_exit，97 行會刪除字符設備，98 行會註銷字符

設備號，最後印出 Goodbye demo!!表示此 module 已被移除。

```
user@user-virtual-machine:~/driver$ sudo insmod demo.ko
[sudo] password for user:
user@user-virtual-machine:~/driver$ sudo rmmod demo
```

接下來進入 terminal insmod 這個 module 以及 rmmod 這個 module。

```
[ 4436.272822] MAJOR = 90 MINOR = 40
[ 4436.272825] Hello demo!!
[ 4445.208751] Goodbye demo!!
```

由 sudo dmesg 可以看到這個 module 被成功的執行當 insmod 執行完會印出 MAJOR = 90 MINOR = 40 以及 Hello demo!!，而當 rmmod 執行完會印出 Goodbye demo!!。

```
user@user-virtual-machine:~/driver$ sudo mknod /dev/demo c 90 40
user@user-virtual-machine:~/driver$ ls -l /dev/demo
crw-r--r-- 1 root root 90, 40  四  20 00:18 /dev/demo
```

接下來使用 sudo mknod /dev/demo c 90 40 建立檔案節點，再由 ls -l /dev/demo 檢查節點是否建立成功。

```
user@user-virtual-machine:~/driver$ cat /proc/devices
Character devices:
  1 mem
  4 /dev/vc/0
  4 tty
  4 ttyS
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  5 ttyprintk
  6 lp
  7 vcs
 10 misc
 13 input
 14 sound/midi
 14 sound/dmmidi
 21 sg
 29 fb
 89 i2c
 90 SenTsai
 99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
202 cpu/msr
204 ttyMAX
226 drm
241 hidraw
242 ttyDBC
243 bsg
244 watchdog
245 remoteproc
246 ptp
247 pps
248 rtc
249 dma_heap
250 dax
251 dimmctl
252 ndctl
253 tpm
254 gpiochip
261 accel
```

最後使用 cat /proc/devices 檢查字符設備是否有被註冊，可以看到 DRIVER_NAME 為 SenTsai 已成功被註冊，主設備號為 90。

Part 2: Write a C based program to access the driver

1. Explain the detailed implementation of your approach.

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<unistd.h>
4 #include<fcntl.h>
5 #include<string.h>
6
7
8 int main()
9 {
10     char *to_driver ="Prepare to write";
11     int size = strlen(to_driver);
12     char to_user[size];
13
14     int fd = open("/dev/demo", O_RDWR);
15     ssize_t ret;
16     if (fd == -1)
17     {
18         printf("Can not open file\n");
19         return -1;
20     }
21     ret = write(fd, to_driver, size);
22     if (ret > 0)
23         printf("Write fail\n");
24
25     ret = read(fd, &to_user[0], size);
26     if (ret > 0)
27         printf("Read fail\n");
28     printf("%s\n", to_user);
29     close(fd);
30
31     return 0;
32 }
33
```

第 10 行宣告一個指標變數 to_driver 並且初始化為 Prepare to write，主要是寫入 driver 的字串，第 11 行宣告了整數變數 size，用來儲存 to_driver 的字串大小，以便後面呼叫 function 時決定需寫入讀取的字元數量，第 12 行宣告了一個字串 to_user，且大小為剛剛取得的 size 大小。

第 14 行宣告一個整數變數 fd 用來儲存呼叫 open function 的回傳值，open function 裡面的/dev/demo 是開啟的節點位置，O_RDWR 是 open function 的標誌，代表開啟的檔案/dev/demo 可讀可寫，16-20 行為處理檔案開啟失敗的狀況且印出 Can not open file，第 15 行宣告了一個變數 ret，用於儲存 read 跟 write 時未複製成功的字元數。

21 行呼叫 write 函式，開啟節點 fd 並且執行其功能，並且將 to_driver 的字串內容寫入 driver 緩存，寫入的字數是 size，若當 ret 大於 0 代表有字元未成功寫入則會執行 22、23 行並且印出 Write fail。

25 行呼叫 read 函式，開啟節點 fd 並且執行其功能，用&取得 to_user[0]的位址然後將 driver 內的字串寫入到 to_user 裡，寫入的字數是 size，若當 ret 大於 0 代表有字元未成功寫入則會執行 26、27 行並且印出 Read fail。

27 行將 to_user 所存的字串印出來，最後 28 行關閉節點 fd。

```
user@user-virtual-machine:~/usr$ gcc main.c -o main
user@user-virtual-machine:~/usr$ sudo chmod 666 /dev/demo
```

首先用 gcc main.c -o main 對 main.c 進行編譯並且產生名為 main 的執行檔，接下來透過 sudo chmod 666 /dev/demo 更改/dev/demo 的讀寫權限，666 表示權限位，其中第一個數字表示所有者的權限，第二個數字表示同組用戶的權限，第三個數字表示其他用戶的權限，數字 6 代表讀取和寫入權限。這意味著擁有該權限的用戶可以讀取和修改文件或設備，透過執行 sudo chmod 666 /dev/demo 命令，你允許任何使用者以讀寫模式訪問 /dev/demo 設備文件，

這樣就能在使用者空間程式中開啟並與設備進行通訊了

```
user@user-virtual-machine:~/usr$ ./main
Prepare to write
```

最後使用./main 執行執行檔 main，最後印出 Prepare to write，完成字串在 driver 進行讀寫的動作。